

Représentation d'un entier positif

Pour mieux appréhender les autres systèmes de numération, commençons par examiner celui que nous utilisons couramment : le système décimal, ou base 10. Pourquoi l'appelle-t-on ainsi ? Tout simplement parce qu'il repose sur **dix chiffres** :

0, 1, 2, 3, 4, 5, 6, 7, 8, 9.

Une fois ces dix chiffres utilisés, il n'y en a plus de disponibles. Pour continuer à compter, on recommence à **zéro** en ajoutant un **nouveau chiffre à gauche**, représentant un rang supérieur — les **dizaines**.

Cela donne :

10, 11, 12, ..., jusqu'à 19.

Après 19, plus de chiffres pour les unités, on incrémente donc encore les dizaines :

20, 21, et ainsi de suite.

Ce processus se poursuit jusqu'à 99. Une fois toutes les combinaisons de dizaines et d'unités épuisées, on ajoute un troisième chiffre à gauche : **100**.

Chaque fois que l'on franchit un seuil de 10, on passe à un **nouveau rang**, ce qui reflète bien le principe de la base 10 : **chaque position correspond à une puissance de 10**.

Par exemple, le nombre **216**, en base 10 (noté $(216)_{10}$), se décompose ainsi :

$$2 \times 10^2 + 1 \times 10^1 + 6 \times 10^0.$$

Le système binaire

Le système binaire est une méthode de représentation des nombres qui n'utilise que **deux chiffres** : **0 et 1**.

C'est le langage de base utilisé par tous les appareils numériques modernes : ordinateurs, téléphones, tablettes, etc.

Pourquoi ? Parce qu'ils fonctionnent à l'électricité, et que la manière la plus simple et fiable de transmettre de l'information dans un circuit électronique repose sur **deux états** :

- **le courant passe** (représenté par **1**)
- **le courant ne passe pas** (représenté par **0**)

Ainsi, chaque donnée traitée par un ordinateur est traduite sous forme de **0 et 1**, permettant à la machine de comprendre et d'exécuter les instructions.

Historiquement, les nombres apparaissent en France en 1600. Jean Neper (1550-1617)



et Gottfried Wilhelm Leibniz (1646-1716) deux grands mathématiciens dont vous entendrez parler si vous suivez des études scientifiques) se sont particulièrement intéressés à ces nombres. Leibniz fut le premier à vouloir concevoir une calculatrice mécanique reposant sur les nombres binaires....comme nos ordinateurs! A une époque où l'électronique n'existait pas!



George Boole (1815-1864) publie "l'algèbre de Boole" en 1847.



Construire des nombres en binaire

Même si le système binaire ne contient que **deux chiffres** (0 et 1), on peut y construire des nombres de la **même manière** qu'en base 10. La seule différence, c'est que dès qu'on a utilisé tous les chiffres disponibles (ce qui arrive très vite avec seulement 0 et 1), on ajoute une **nouvelle colonne à gauche** pour continuer à compter.

Comptons ensemble en binaire :

```
0
1 → Plus de chiffres disponibles !
10 → On remet à zéro et on ajoute un 1 à gauche
11
100
101
110
```

Comme en base 10, chaque nouvelle colonne correspond à un rang de puissance — ici des puissances de 2.

Astuce : On ne progresse pas en lisant uniquement... il faut pratiquer ! Essayez vous-même de continuer la table ci-dessus.

Pour aller plus loin, amusez-vous à compter en **base 3** (avec les chiffres 0, 1 et 2), ou en **base 7** (de 0 à 6), pour mieux saisir le fonctionnement des systèmes de numération.

Voici la **correspondance** entre les écritures binaires et leurs équivalents en base décimale :

Binaire	Décimal
0	0
1	1
10	2
11	3
100	4
101	5
110	6

Qu'est-ce qu'un bit ?

Vous avez sans doute déjà entendu parler des "**bits**", notamment en lien avec les consoles de jeux vidéo. Par exemple, les premières consoles comme l'**Atari 2600** (1977, États-Unis) ou la **Nintendo Entertainment System (NES)** (1983) étaient des consoles **8 bits**. Par la suite, sont apparues les consoles **16 bits** comme la **Sega Mega Drive** ou la **Super Nintendo (SNES)**. Aujourd'hui, certaines machines atteignent des architectures **128 bits**.

Mais que signifie ce terme "**bit**" ?

Le **Binary Digit** dont la contraction donne **bit** est l'unité de base en informatique. Il peut prendre **deux valeurs : 0 ou 1**, ce qui correspond à deux états physiques dans un circuit électronique :

- le courant passe → 1
- le courant ne passe pas → 0

Autrement dit, un **bit est un chiffre en binaire**. Et comme nous l'avons vu, en combinant plusieurs bits, on peut former des **nombres binaires**, tout comme on forme des nombres en base 10 à partir des chiffres de 0 à 9.



Combien de valeurs peut-on représenter avec des bits ?

Reprenons notre exemple : pour écrire le nombre **4** en binaire, il a fallu **3 bits**. Cela montre que **plus on a de bits, plus on peut représenter de valeurs différentes**.

Alors, combien de valeurs peut-on écrire avec :

- une **vieille NES** ou une **Sega Mega Drive** (8 bits, 16 bits) ?
- ou encore avec une **PS5** qui utilise des architectures bien plus avancées, comme le **128 bits** ?

Rappel :

1 octet = 8 bits



Cela signifie qu'avec **8 bits**, on peut représenter $2^8 = 256$ **valeurs différentes**, allant de **0** à **255**.

Quelques rappels utiles sur les unités de stockage :

- **1 To (téraoctet) = 1000 Go (gigaoctets)**
- **1 Go = 1000 Mo (mégaoctets)**
- **1 Mo = 1000 Ko (kiloctets) = 1 000 000 octets**

En programmation, les **nombre entiers** sont généralement appelés des **"integers"** (ou **int**).

Convertir un nombre binaire en décimal

Pour convertir un nombre de la base binaire (base 2) en base décimale (base 10), on utilise la même logique que celle employée pour les nombres en base 10 :

Chaque chiffre est multiplié par une **puissance de la base**, selon sa position.

Exemple :

Le nombre binaire **(101)₂** se décompose ainsi :

$$1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 4 + 0 + 1 = 5 \text{ en base 10}$$

Formule générale pour toute base b :

Si un nombre s'écrit **(xyzw)_(b)**, alors :

$$(xyzw)_{(b)} = x \times b^3 + y \times b^2 + z \times b^1 + w \times b^0$$

Cela fonctionne pour n'importe quelle base :

- **b = 2** pour le **binaire**
- **b = 10** pour le **décimal**
- etc.

Convertir un nombre décimal en binaire

Comme nous l'avons vu précédemment, écrire un nombre dans une base consiste à utiliser les **puissances successives de cette base**, en commençant par la plus grande possible, puis en traitant ce qui reste avec les puissances inférieures.

Dans le cas du **passage du décimal au binaire** (c'est-à-dire de la base 10 à la base 2), on utilise une méthode simple : faire des **divisions successives par 2**.

À chaque étape, on note **le reste** (soit 0, soit 1), qui correspond à un **bit** du résultat binaire.

Exemple : convertir 216 en binaire

$216 \div 2 = 108$	reste 0
$108 \div 2 = 54$	reste 0
$54 \div 2 = 27$	reste 0
$27 \div 2 = 13$	reste 1
$13 \div 2 = 6$	reste 1
$6 \div 2 = 3$	reste 0
$3 \div 2 = 1$	reste 1
$1 \div 2 = 0$	reste 1

Ensuite, **on lit les restes de bas en haut** :
 $(216)_{10} = (11011000)_2$

Astuce : Cette méthode fonctionne pour convertir n'importe quel entier décimal en binaire. Elle peut être généralisée à d'autres bases en remplaçant la division par 2 par une division par la base cible (par exemple, division par 3 pour la base 3, ou par 16 pour l'hexadécimal).

Voici une table binaire sur 8 bits en largeur, avec les puissances de 2 indiquées pour chaque position de bit, de gauche à droite :

Poids des bits (puissances de 2)

Bit n°	7	6	5	4	3	2	1	0
Puissance	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Valeur décimale	128	64	32	16	8	4	2	1

Exemple avec le nombre binaire 11001010 :

Bit	1	1	0	0	1	0	1	0	
Poids (2^n)	128	64	32	16	8	4	2	1	
Produit	128	64	0	0	8	0	2	0	

$$\text{Somme} = 128 + 64 + 8 + 2 = 202$$

Donc :

$$(11001010)_2 = (202)_{10}$$

Le système hexadécimal

Même si l'ordinateur fonctionne en **binaire** (avec des 0 et des 1), cette écriture n'est **pas très lisible pour nous**, les humains. Heureusement, il existe un autre système plus pratique pour **représenter le binaire de manière compacte** : c'est le **système hexadécimal**, ou **base 16**.

Qu'est-ce que la base 16 ?

Comme son nom l'indique, la base 16 utilise **16 symboles** pour écrire les nombres. Or, nous ne disposons que de **10 chiffres classiques** (de 0 à 9), donc pour les six suivants, on utilise **des lettres** :

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

Cela nous permet de compter ainsi :

...9, A, B, C, D, E, F, puis 10 (qui correspond à 16 en base 10), 11, 12, ..., 1A, 1B, etc.

Pourquoi utiliser l'hexadécimal ?

Parce que le système hexadécimal est **étroitement lié au binaire** :

1 chiffre hexadécimal = 4 bits (exactement)

En effet, $2^4=16$

Cela signifie qu'un **nombre binaire de 8 bits** peut être représenté par **2 chiffres hexadécimaux seulement**.

Exemple de conversion :

Prenons le nombre binaire suivant :

(01011011)₂

On le regroupe en **paquets de 4 bits**, en partant de la droite :

→ (0101 1011)₂

Chaque groupe de 4 bits correspond à un chiffre hexadécimal :

- 0101 = 5
- 1011 = B

Donc :

(01011011)₂ = (5B)₁₆

Astuce : Toujours regrouper les bits par groupes de 4 en partant de la droite, et ajouter des zéros à gauche si nécessaire pour compléter le groupe.

Exemples :

- **(4D)₁₆** → 4 = 0100, D = 1101 → **(01001101)₂**
- **(D4)₁₆** → D = 1101, 4 = 0100 → **(11010100)₂**

Erreur fréquente : Ne pas regrouper correctement les bits peut conduire à une mauvaise conversion.

Par exemple, ne pas écrire $(D4)_{16}$ comme $(1101100)_2$, ce serait incorrect.

Conversion hexadécimal → décimal

Pour convertir un nombre hexadécimal en base 10, on utilise les **mêmes méthodes** que pour les autres bases :

- **Multiplier chaque chiffre** par une **puissance de 16**, selon sa position.
- Ou effectuer des **divisions successives par 16** (division euclidienne), comme on l'a fait pour le binaire.

Table de correspondance binaire ↔ hexadécimal :

Binaire	Hexa
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F